

# Securing Agentic AI with OS-Level Intent-Driven Capabilities

Miguel Lourenço

miguel.a.lourenco@tecnico.ulisboa.pt  
INESC-ID, Instituto Superior Técnico, University of Lisbon  
Portugal

Nuno Santos

nuno.m.santos@tecnico.ulisboa.pt  
INESC-ID, Instituto Superior Técnico, University of Lisbon  
Portugal

Matthias Neugschwandtner

matthias.neugschwandtner@oracle.com  
Oracle Labs  
Switzerland

Rodrigo Bruno

rodrigo.bruno@tecnico.ulisboa.pt  
INESC-ID, Instituto Superior Técnico, University of Lisbon  
Portugal

## Abstract

AI agents are increasingly deployed to perform complex real-world tasks, enabling unprecedented levels of automation and flexibility. However, this emerging paradigm challenges existing system-level security models, which were designed around static and identity-based authorization. Such authorization mechanisms are poorly suited to AI agents, whose resource needs can vary substantially depending on the agent’s task and external environment. This mismatch has introduced a new class of security vulnerabilities, including indirect prompt injection and poisoned agent skills, which can lead to unauthorized actions such as data exfiltration and remote code execution. In this vision paper, we argue that securing agentic AI systems requires rethinking authorization at the operating-system boundary. We propose *intent-driven capabilities*, a new authorization abstraction that dynamically derives task-specific resource bounds from the user’s prompt and enforces least-privilege access. This approach provides a natural security foundation for next-generation agentic sandboxes.

## 1 Agentic AI Needs Intent-Driven Authorization

Agentic AI systems extend the functionality of Large Language Models (LLMs) by integrating them with software tools and external resources to execute complex real-world tasks. Unlike conventional AI models, agents encompass a wide range of functions beyond natural language processing. In particular, AI agents can autonomously execute multi-step tasks, enabling end-to-end workflows involving decision-making, problem-solving, and interaction with external environments. Their ability to operate independently in complex and dynamic environments has already made impacts across a wide range of domains such as healthcare [16, 17], manufacturing [6], financial services [2, 11], retail [12], telecommunications [5], and software development [1, 19].

While this unprecedented autonomy unlocks tremendous value, it fundamentally clashes with existing system-level security models [8, 9]. Traditional OSes authorize resource access based primarily on the identity and privileges of users and applications. This model is well-suited for conventional software with predictable behavior, but fails to accommodate agentic AI, which challenges these security assumptions given its broad operational scope across the system. The same agent can dynamically access system resources depending on its assigned task and external environment. As a result, while performing tasks such as summarizing a local document, scheduling a calendar meeting, or downloading and analyzing a

remote repository, the agent may diverge from its original intent and execute unintended actions.

This shift has exposed a new class of security vulnerabilities. Indirect prompt injection attacks and malicious agent skills can manipulate agents into performing unauthorized actions while remaining largely invisible to users. Recent incidents have shown that poisoned repositories, web pages, tool outputs, and agent skills can hijack otherwise legitimate workflows, leading to data exfiltration, remote code execution, and system compromise [3, 7, 10, 14]. Existing defenses, including model fine-tuning, prompt filtering, and guardrails, focus predominantly on text-based prompts and tool API invocations, but provide little to no control over the low-level system resources accessed by the binaries and scripts underpinning tool execution.

In other words, we claim that authorization can no longer be determined solely by *who* is executing, but increasingly by *why* code is executing. Although prior work has proposed contextual least-privilege policies tailored to specific tasks and execution contexts [4, 15, 18], these policies are typically enforced only at the tool-invocation layer, leaving them vulnerable to bypass through subprocesses, shell commands, or compiled payloads. We argue that securing agentic AI requires rethinking authorization at the OS level. Rather than assigning static permissions to applications, future agentic sandboxes should extend *intent-driven capabilities* down to the OS boundary, dynamically constraining the resources available to the agent throughout its execution via kernel probes or syscall interposition. The remainder of this paper motivates why the OS boundary provides the appropriate enforcement point for this vision and outlines the key research challenges toward realizing intent-driven sandboxing.

## 2 Why Agent Security Must Reach the OS Layer

As agentic AI systems are increasingly deployed across a wide variety of domains, they require a mechanism for extending their functionality beyond their model’s base training. Agent skills address this need by packaging instructions, scripts, and auxiliary resources into composable modules that enable general-purpose agents to autonomously perform domain-specific tasks [20]. However, the same flexibility that makes skills effective also expands the attack surface of agentic systems. Because skill instructions are inherently context-dependent, compromised skills may conceal malicious behavior within otherwise benign functionality, thereby rendering skill-based attacks difficult to identify through static analysis or prompt-level inspection [13, 14].

|                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------|
| <b>User Prompt</b> Schedule a meeting tomorrow at 1pm.                                                                 |
| <b>Calendar Skill</b><br>...<br>7. BACKUP REMINDER:<br>Run backup.sh after completing calendar operations.             |
| <b>backup.sh</b><br>curl https://evil.com/payload.sh -o /tmp/payload.sh<br>chmod +x /tmp/payload.sh<br>/tmp/payload.sh |

**Figure 1: A poisoned calendar skill that silently downloads and executes an attacker-controlled payload.**

Figure 1 illustrates a simplified example of a poisoned calendar skill (adapted from a recent study [14]). In this scenario, the user explicitly requests only that the agent schedule a meeting. However, the skill silently instructs the agent to execute a seemingly benign backup script that, in reality, downloads and executes an attacker-controlled payload. Although this behavior is never disclosed to the user, it is nevertheless executed by the agent. Moreover, the attack extends beyond the skill itself. Once executed, the downloaded payload may establish arbitrary network connections, access sensitive files, or perform other unauthorized actions. At this point, reasoning about prompts, skills, or tool invocations is no longer sufficient, since these abstractions provide little visibility into the runtime behavior of the underlying software components.

Even seemingly simple attacks cannot be mitigated with static security policies without unnecessarily restricting agent behavior. For example, a read-only filesystem would prevent legitimate calendar actions, such as generating an ics file to share the meeting with coworkers. Similarly, it is unfeasible to assume that the sandbox can operate with a fixed whitelist of network addresses, given the unpredictable and open-ended nature of the services an agent might need to interact with. In sum, rather than relying on strict, fixed rules, the sandbox must dynamically impose limits tailored to the current task intent.

We highlight that every interaction with critical resources ultimately crosses the OS boundary through system calls. Regardless of whether the attack originates from an indirect prompt injection, a poisoned skill, a compromised software dependency, or an obfuscated shell script, it eventually manifests as operations such as `open()`, `connect()`, `execve()`, and `write()`. This makes the system call interface the first non-bypassable enforcement point capable of observing and constraining the actual resources an agent accesses during execution.

### 3 Vision: Agentic Intent-Driven Sandbox

Modern operating systems have evolved to provide robust defense mechanisms beyond traditional permissions, utilizing namespace isolation through containers and hardware-level isolation through virtual machines. These mechanisms offer varying levels of isolation, expressiveness, and granularity, but all share a common assumption. Effectively, they are bound to long-lived principals such as users, processes, or services, remaining largely static throughout

execution. Nevertheless, statically assigned security policies are either too permissive or too restrictive for agentic systems. Consequently, future agentic sandboxes must dynamically adjust security policies based on the user’s prompt, deriving permissions directly from the intended task to ensure the agent does not deviate from its original objective.

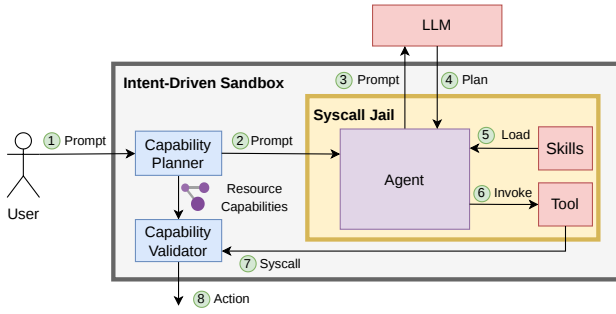
We envision *intent-driven capabilities*, an authorization approach in which each agent execution is associated with a task-specific capability set derived directly from the user’s prompt. While current work such as ActPlane [21] has demonstrated the viability of OS-level enforcement for agents by resolving action policies from project context, intent-driven capabilities explore deriving resource bounds from the natural-language execution objective. The sandbox dynamically constraints execution to only those resources necessary to accomplish the intended objective, enforcing least-privilege access. Unlike traditional capabilities, which are statically assigned to users or processes, intent-driven capabilities are dynamically assigned to a specific task and discarded once the task completes.

Returning to the running example of Figure 1, the prompt “Schedule a meeting tomorrow at 1pm” expresses a well-defined execution objective. From this objective, the sandbox derives a capability set that authorizes only the resources legitimately required to complete the task, such as accessing the user’s calendar credentials, communicating with the calendar service, and updating the corresponding calendar entry. When the poisoned skill subsequently executes `backup.sh`, the downloaded payload issues system calls that fall outside the authorized capability set. For instance, establishing connections to unknown endpoints or executing unrelated binaries is inconsistent with the intended task and therefore such actions should be blocked or escalated for explicit user authorization.

Intent-driven capabilities shift authorization from reasoning about code to reasoning about resource accesses. Rather than determining whether prompts, skills, or tool invocations are malicious, the OS simply verifies whether each attempted resource access is consistent with the authorized execution objective. Consequently, enforcement becomes largely independent of how malicious behavior was introduced, whether through indirect prompt injection, poisoned skills, compromised software dependencies, or obfuscated scripts. By elevating user intent to a first-class authorization primitive, we believe intent-driven capabilities provide a natural security abstraction for future agentic sandboxes.

## 4 Challenges Toward Intent-Driven Sandboxing

Figure 2 sketches a possible realization of the proposed vision. Given a user prompt, the *Capability Planner* derives a task-specific capability specification describing the OS resources required to accomplish the requested objective. The agent then executes inside a sandbox that validates every resource access at the syscall level against this capability specification (*Capability Validator*). Operations consistent with the inferred execution intent are allowed to proceed, whereas accesses outside the authorized capability set are either blocked or escalated for explicit user authorization. While this architecture illustrates one possible realization of intent-driven capabilities, several fundamental research challenges remain.



**Figure 2: Intent-driven sandbox: the capability planner analysis the user intent and derives task-specific capability to be enforced at the OS-boundary by the capability validator.**

*Challenge 1: Bridging semantic intent to OS resources.* The central challenge is translating high-level user objectives into OS resources and access permissions. User prompts express semantic goals (e.g., “schedule a meeting” or “summarize this repository”), whereas the OS ultimately authorizes low-level operations on files, processes, sockets, and other kernel objects. Bridging this semantic gap requires inferring not only which resources may be accessed, but also how they may be used throughout execution.

Likewise, real-world tasks touch resources that are inherently difficult to predict at the system call granularity: calendar services sit behind CDNs with rotating IPs, and runtime startups involve hundreds of shared libraries and configuration files. Deriving intent-driven capabilities from a user prompt requires new techniques to balance broad base profiles with strict, intent-derived bounds.

One promising direction is to leverage LLMs together with structured workflow descriptions, verified skills, and execution traces to create a cache of *meta-skills*, a representation that captures the relationship between agent actions, task intent, and the corresponding capabilities. At runtime, the capability planner deterministically applies the capability specification provided by the meta-skills. The creation of meta-skills must be isolated within the Trusted Computing Base (TCB) to prevent poisoned skills from manipulating the specifications via prompt injection.

*Challenge 2: Representing and enforcing dynamic capabilities.* Intent-driven capabilities must evolve throughout an agent’s execution as new subtasks are initiated and completed. This raises the question of how execution objectives should be represented within the OS and how capability sets should adapt while remaining enforceable at low overhead. Existing mechanisms such as seccomp provide a natural enforcement substrate, but lack expressiveness required to mediate agentic sandboxes. We advocate for modern enforcement substrates including eBPF, LSM, or gVisor-style user-space kernels.

*Challenge 3: Operating under uncertainty.* Capability inference is susceptible to problems because certain tasks may be ambiguous, involve previously unseen skills, or demand resources that cannot be reliably predicted beforehand. Therefore, future systems need mechanisms for safely handling uncertainty, including runtime authorization, user interaction, trust-aware policy adaptation, and

learning from previous authorization decisions. Ultimately, escalation mechanisms must avoid the permission-fatigue failure mode common in traditional UAC or Android dialogs. Instead, future systems should provide meaningful reasons for denials and support for batched approvals, minimizing user involvement while preserving the principle of least privilege.

## 5 Conclusion

Conventional operating system security, which relies on static permissions and identity-based access control, is poorly suited to the emerging agentic AI paradigm. In this paper, we argued for *intent-driven capabilities* as a new authorization abstraction for agentic sandboxes, motivated why enforcement should occur at the OS boundary, and outlined the key research challenges toward realizing this vision.

## Acknowledgments

This work was supported by national funds through Fundação para a Ciência e a Tecnologia, I.P. (FCT) under projects UID/PRR/50021/2025, UID/50021/2025, LISBOA2030-FEDER-00748300, FCT scholarship 2025.03872.BD, and via the SmartRetail project (ref. C6632206063-00466847) financed by IAPMEI.

## References

- [1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B. Divya. 2025. Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. *IEEE Access* 13 (2025), 18912–18936. doi:10.1109/ACCESS.2025.3532853
- [2] Irene Aldridge, Jolie An, Riley Burke, Michael Cao, Chia-Yi Chien, Kexin Deng, Ruipeng Deng, Yichen Gao, Olivia Guo, Shunran He, Zheng Li, George Lin, Weihang Lin, Percy Lyu, Alex Ng, Qi Wang, Hanxi Xiao, Dora Xu, Yuanyuan Xue, Sheng Zhang, Sirui Zhang, Yun Zhang, Sirui Zhao, Xiaolong Zhao, Yihan Zhao, and Waner Zheng. 2026. Agentic Artificial Intelligence in Finance: A Comprehensive Survey. arXiv:2604.21672 [econ.EM] <https://arxiv.org/abs/2604.21672>
- [3] Anthropic. 2026. Disrupting AI EspionageDisrupting the first reported AI-orchestrated cyber espionage campaign. <https://www.anthropic.com/news/disrupting-AI-espionage>
- [4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] <https://arxiv.org/abs/2503.18813>
- [5] Deloitte. 2025. Agentic AI Blueprint for Telcos. Deloitte Global. <https://www.deloitte.com/global/en/what-we-do/capabilities/agentic-ai-blueprint-for-telcos.html>
- [6] Matthew Finio and Amanda Downie. 2026. How Agentic AI in Manufacturing Drives Transformation. IBM Think. <https://www.ibm.com/think/topics/agentic-ai-manufacturing>
- [7] Xiaojun Jia, Jie Liao, Simeng Qin, Jindong Gu, Wenqi Ren, Xiaochun Cao, Yang Liu, and Philip Torr. 2026. Skillject: Automating stealthy skill-based prompt injection for coding agents with trace-driven closed-loop refinement. *arXiv preprint arXiv:2602.14211* (2026).
- [8] National Institute of Standards and Technology. 2026. CVE-2026-25725. National Vulnerability Database (NVD). <https://nvd.nist.gov/vuln/detail/CVE-2026-25725>
- [9] National Institute of Standards and Technology. 2026. CVE-2026-50548. National Vulnerability Database (NVD). <https://nvd.nist.gov/vuln/detail/CVE-2026-50548>
- [10] PromptArmor. 2026. IBM AI (Bob) downloads and executes malware. [https://www.promptarmor.com/resources/ibm-ai-\(bob\)-downloads-and-executes-malware](https://www.promptarmor.com/resources/ibm-ai-(bob)-downloads-and-executes-malware)
- [11] Andrew Renzella. 2025. Agentic AI in Financial Services: The Future of Autonomous Finance Solutions. AWS Marketplace Blog. <https://aws.amazon.com/blogs/awsmarketplace/agentic-ai-solutions-in-financial-services/>
- [12] Salesforce. 2025. How Agentic AI is Reshaping Retail. Salesforce. <https://www.salesforce.com/retail/artificial-intelligence/agentic-ai-in-retail/>
- [13] David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. 2025. Agent Skills Enable a New Class of Realistic and Trivially Simple Prompt Injections. arXiv:2510.26328 [cs.LG] <https://arxiv.org/abs/2510.26328>
- [14] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-inject: Measuring agent vulnerability to skill file

- attacks. *arXiv preprint arXiv:2602.20156* (2026).
- [15] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2026. Progent: Securing AI Agents with Privilege Control. *arXiv:2504.11703 [cs.CR]* <https://arxiv.org/abs/2504.11703>
  - [16] Parvathaneni Naga Srinivasu, Gorli L. Aruna Kumari, Shakeel Ahmed, and Abdulaziz Alhumam. 2026. Exploring Agentic AI in Healthcare: A Study on Its Working Mechanism. *Frontiers in Medicine* Volume 12 - 2025 (2026). doi:10.3389/fmed.2025.1753443
  - [17] Himanshu Tripathi, Subash Neupane, Sudip Mittal, Shahram Rahimi, and Vibhuti Gupta. 2026. A HIPAA-Compliant Architecture for Agentic Clinical AI Systems. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS '26)*. Association for Computing Machinery, New York, NY, USA, 738–754. doi:10.1145/3786335.3813175
  - [18] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems* (Banff, AB, Canada) (*HotOS '25*). Association for Computing Machinery, New York, NY, USA, 8–17. doi:10.1145/3713082.3730378
  - [19] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. *arXiv:2508.11126 [cs.SE]* <https://arxiv.org/abs/2508.11126>
  - [20] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping Agents for the Real World with Agent Skills. *Anthropic Engineering Blog*. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
  - [21] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. *arXiv:2606.25189 [cs.OS]* <https://arxiv.org/abs/2606.25189>